

```
1  /* =====
2      CELEBAL TECHNOLOGIES
3      DATA ENGINEERING INTERNSHIP
4
5      WEEK 3 ASSIGNMENT
6
7  Dataset : Superstore
8  Tool    : SQL Server Management Studio (SSMS)
9
10 Topic   : Subqueries, CTEs & Window Functions
11 ===== */
12
13 /*-----
14 STEP 1 : SETUP DATA
15 Task 1 : Create Database
16 -----*/
17 CREATE DATABASE SuperstoreDB;
18 GO
19 USE SuperstoreDB;
20 GO
21
22 /*-----
23 Task 2
24
25 Import the Superstore dataset into a table named
26 superstore_raw using the SSMS Import Flat File Wizard.
27 -----*/
28
29 -- Verify the imported data
30 SELECT COUNT(*) AS total_rows FROM superstore_raw;
31 SELECT TOP 5 * FROM superstore_raw;
32
33
34 /*-----
35 Task 3
36
37 Create normalized tables:
38 1. customers
39 2. orders
40 3. products
41 -----*/
42
43 -- Create Customers Table
44 CREATE TABLE customers (
45     customer_id  VARCHAR(20) PRIMARY KEY,
46     customer_name VARCHAR(100),
47     segment      VARCHAR(30)0),
48     country      VARCHAR(50)
49 );
```

```
50
51 INSERT INTO customers (customer_id, customer_name, segment, country)
52 SELECT DISTINCT Customer_ID, Customer_Name, Segment, Country
53 FROM superstore_raw;
54
55 -- Create Products Table
56 -- Some Product_ID values appear with multiple names in the dataset.      ↗
   ROW_NUMBER() is used to keep one record for each Product_ID.
57 CREATE TABLE products (
58     product_id  VARCHAR(30) PRIMARY KEY,
59     category    VARCHAR(30),
60     sub_category VARCHAR(30),
61     product_name VARCHAR(255)
62 );
63
64 INSERT INTO products (product_id, category, sub_category, product_name)
65 SELECT product_id, category, sub_category, product_name
66 FROM (
67     SELECT
68         Product_ID AS product_id,
69         Category   AS category,
70         Sub_Category AS sub_category,
71         Product_Name AS product_name,
72         ROW_NUMBER() OVER (PARTITION BY Product_ID ORDER BY Product_Name) AS rn
73     FROM superstore_raw
74 ) t
75 WHERE rn = 1;
76
77 -- Create Orders Table (one row per order line, with shipping/location    ↗
   detail)
78 CREATE TABLE orders (
79     row_id      INT PRIMARY KEY,
80     order_id    VARCHAR(20),
81     order_date  DATE,
82     ship_date   DATE,
83     ship_mode   VARCHAR(30),
84     customer_id VARCHAR(20),
85     product_id  VARCHAR(30),
86     city        VARCHAR(50),
87     state       VARCHAR(50),
88     postal_code  VARCHAR(10),
89     region      VARCHAR(20),
90     sales       DECIMAL(10,2),
91     quantity    INT,
92     discount    DECIMAL(4,2),
93     profit      DECIMAL(10,2)
94 );
95
```

```
96 INSERT INTO orders
97 SELECT
98     Row_ID, Order_ID, Order_Date, Ship_Date, Ship_Mode,
99     Customer_ID, Product_ID, City, State, Postal_Code, Region,
100     Sales, Quantity, Discount, Profit
101 FROM superstore_raw;
102
103 -- Row-count check
104 SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
105 UNION ALL
106 SELECT 'products', COUNT(*) FROM products
107 UNION ALL
108 SELECT 'orders', COUNT(*) FROM orders;
109
110
111 /* -----
112     STEP 3: Subqueries
113     ----- */
114
115 -- Query 1 : Find all orders where sales are greater than the average      ↗
116     sales: Baseline - overall average sales value
117 SELECT ROUND(AVG(sales), 2) AS avg_sales
118 FROM orders;
119
120 -- Query 2 : Find the highest sales order for each customer: Orders with    ↗
121     Sales above the overall average (simple subquery)
122 SELECT o.order_id, c.customer_name, p.product_name, o.sales
123 FROM orders o
124 JOIN customers c ON o.customer_id = c.customer_id
125 JOIN products p ON o.product_id = p.product_id
126 WHERE o.sales > (SELECT AVG(sales) FROM orders)
127 ORDER BY o.sales DESC;
128
129 --2. Highest sales order for each customer
130 -- Query 3 : Calculate total sales for each customer: Highest single      ↗
131     order value per customer (correlated subquery)
132 SELECT o.customer_id, c.customer_name, o.order_id, o.sales AS             ↗
133     highest_order_sales
134 FROM orders o
135 JOIN customers c ON o.customer_id = c.customer_id
136 WHERE o.sales = (
137     SELECT MAX(o2.sales)
138     FROM orders o2
139     WHERE o2.customer_id = o.customer_id
140 )
141 ORDER BY o.sales DESC;
142
143 /* -----
```

```
141 STEP 4: CTEs (Common Table Expressions)
142 ----- */
143 -- 3. Total sales for each customer (CTE)
144 -- Query 4 : Find customers whose total sales are above average: Total sales per customer (CTE)
145 WITH customer_sales AS (
146     SELECT c.customer_id, c.customer_name,
147            SUM(o.sales) AS total_sales,
148            COUNT(DISTINCT o.order_id) AS total_orders
149     FROM orders o
150     JOIN customers c ON o.customer_id = c.customer_id
151     GROUP BY c.customer_id, c.customer_name
152 )
153 SELECT *
154 FROM customer_sales
155 ORDER BY total_sales DESC;
156
157 --4. Customers whose total sales are above average (CTE + Subquery)
158
159
160 -- Query 5 : Rank all customers based on total sales: Customers whose TOTAL sales exceed the average customer total
161 -- (CTE combined with a subquery in the WHERE clause)
162 WITH customer_sales AS (
163     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales
164     FROM orders o
165     JOIN customers c ON o.customer_id = c.customer_id
166     GROUP BY c.customer_id, c.customer_name
167 )
168 SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales
169 FROM customer_sales
170 WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales)
171 ORDER BY total_sales DESC;
172
173
174 /* -----
175 STEP 5: Window Functions
176 ----- */
177
178 -- Query 6 : Assign row numbers to each order within a customer: Customer ranking - JOIN + CTE + Window Functions
179 -- (ROW_NUMBER, RANK, DENSE_RANK side by side)
180 WITH customer_sales AS (
181     SELECT c.customer_id, c.customer_name, c.segment,
182            SUM(o.sales) AS total_sales
183     FROM orders o
184     JOIN customers c ON o.customer_id = c.customer_id
185     GROUP BY c.customer_id, c.customer_name, c.segment
186 )
```

```
187 SELECT
188     customer_id,
189     customer_name,
190     segment,
191     ROUND(total_sales, 2) AS total_sales,
192     ROW_NUMBER() OVER (ORDER BY total_sales DESC) AS row_num,
193     RANK() OVER (ORDER BY total_sales DESC) AS sales_rank,
194     DENSE_RANK() OVER (ORDER BY total_sales DESC) AS dense_sales_rank
195 FROM customer_sales
196 ORDER BY total_sales DESC;
197
198 --6. Row numbers for each order within a customer (Window Function +
    PARTITION BY)
199 SELECT o.customer_id, c.customer_name, o.order_id, o.order_date, o.sales,
200     ROW_NUMBER() OVER (PARTITION BY o.customer_id ORDER BY
    o.order_date) AS order_row_num
201 FROM orders o
202 JOIN customers c ON o.customer_id = c.customer_id
203 ORDER BY o.customer_id, order_row_num;
204
205 --7. Top 3 customers based on total sales (Window Function)
206 WITH customer_sales AS (
207     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales
208     FROM orders o JOIN customers c ON o.customer_id = c.customer_id
209     GROUP BY c.customer_id, c.customer_name
210 ),
211 ranked AS (
212     SELECT customer_id, customer_name, ROUND(total_sales, 2) AS
    total_sales,
213     RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
214     FROM customer_sales
215 )
216 SELECT * FROM ranked WHERE sales_rank <= 3;
217
218 -- Final Combined Query (Customer Name, Total Sales, Rank)
219 WITH customer_sales AS (
220     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales
221     FROM orders o
222     JOIN customers c ON o.customer_id = c.customer_id
223     GROUP BY c.customer_id, c.customer_name
224 )
225 SELECT
226     customer_name,
227     ROUND(total_sales, 2) AS total_sales,
228     RANK() OVER (ORDER BY total_sales DESC) AS rank
229 FROM customer_sales
230 ORDER BY rank;
231
232 /* -----
```

```
233 STEP 6: Mini Project: Customer Sales Insights
234 ----- */
235
236 -- Query 7 : Display the Top 3 customers based on total sales: Top 5 customers by total sales
237 WITH customer_sales AS (
238     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales
239     FROM orders o JOIN customers c ON o.customer_id = c.customer_id
240     GROUP BY c.customer_id, c.customer_name
241 ),
242 ranked AS (
243     SELECT customer_id, customer_name, ROUND(total_sales, 2) AS
244         total_sales,
245         RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
246     FROM customer_sales
247 )
248 SELECT * FROM ranked WHERE sales_rank <= 5;
249
250 -- Query 8: Bottom 5 customers by total sales (low-value customers)
251 WITH customer_sales AS (
252     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales
253     FROM orders o JOIN customers c ON o.customer_id = c.customer_id
254     GROUP BY c.customer_id, c.customer_name
255 ),
256 ranked AS (
257     SELECT customer_id, customer_name, ROUND(total_sales, 2) AS
258         total_sales,
259         RANK() OVER (ORDER BY total_sales ASC) AS low_rank
260     FROM customer_sales
261 )
262 SELECT * FROM ranked WHERE low_rank <= 5;
263
264 -- Query 9: Customers who placed only ONE order
265 WITH order_counts AS (
266     SELECT c.customer_id, c.customer_name,
267         COUNT(DISTINCT o.order_id) AS order_count,
268         SUM(o.sales) AS total_sales
269     FROM orders o JOIN customers c ON o.customer_id = c.customer_id
270     GROUP BY c.customer_id, c.customer_name
271 )
272 SELECT customer_id, customer_name, order_count, ROUND(total_sales, 2) AS
273     total_sales
274 FROM order_counts
275 WHERE order_count = 1
276 ORDER BY total_sales DESC;
277
278 --4. Customers with above-average sales
279 WITH customer_sales AS (
280     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales
281     FROM orders o JOIN customers c ON o.customer_id = c.customer_id
```

```
278     GROUP BY c.customer_id, c.customer_name
279 )
280 SELECT customer_id, customer_name, ROUND(total_sales,2) AS total_sales
281 FROM customer_sales
282 WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales)
283 ORDER BY total_sales DESC;
284
285 --5 .Highest order value per customer
286 SELECT o.customer_id, c.customer_name, o.order_id, o.sales AS           ↗
       highest_order_sales
287 FROM orders o
288 JOIN customers c ON o.customer_id = c.customer_id
289 WHERE o.sales = (SELECT MAX(o2.sales) FROM orders o2 WHERE o2.customer_id ↗
       = o.customer_id)
290 ORDER BY o.sales DESC;
291 -- Query 1 : Find all orders where sales are greater than the average   ↗
       sales0: Top 3 customers per Region (PARTITION BY)
292 WITH customer_region_sales AS (
293     SELECT c.customer_id, c.customer_name, o.region, SUM(o.sales) AS     ↗
       total_sales
294     FROM orders o JOIN customers c ON o.customer_id = c.customer_id
295     GROUP BY c.customer_id, c.customer_name, o.region
296 ),
297 ranked AS (
298     SELECT customer_id, customer_name, region, ROUND(total_sales, 2) AS   ↗
       total_sales,
299         RANK() OVER (PARTITION BY region ORDER BY total_sales DESC) AS   ↗
       rank_in_region
300     FROM customer_region_sales
301 )
302 SELECT * FROM ranked WHERE rank_in_region <= 3
303 ORDER BY region, rank_in_region;
304
305 -- Query 1 : Find all orders where sales are greater than the average   ↗
       sales1: Customer segmentation into quartiles (NTILE)
306 WITH customer_sales AS (
307     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales
308     FROM orders o JOIN customers c ON o.customer_id = c.customer_id
309     GROUP BY c.customer_id, c.customer_name
310 ),
311 segmented AS (
312     SELECT customer_id, customer_name, ROUND(total_sales, 2) AS           ↗
       total_sales,
313         NTILE(4) OVER (ORDER BY total_sales DESC) AS quartile
314     FROM customer_sales
315 )
316 SELECT quartile,
317     COUNT(*) AS num_customers,
318     ROUND(AVG(total_sales), 2) AS avg_sales_in_quartile,
```

```
319         ROUND(MIN(total_sales), 2) AS min_sales,
320         ROUND(MAX(total_sales), 2) AS max_sales
321 FROM segmented
322 GROUP BY quartile
323 ORDER BY quartile;
324
325 -- Query 1 : Find all orders where sales are greater than the average sales2: Best-selling product per category (window function)
326 WITH product_sales AS (
327     SELECT p.category, p.product_name, SUM(o.sales) AS total_sales
328     FROM orders o JOIN products p ON o.product_id = p.product_id
329     GROUP BY p.category, p.product_name
330 ),
331 ranked AS (
332     SELECT category, product_name, ROUND(total_sales, 2) AS total_sales,
333         RANK() OVER (PARTITION BY category ORDER BY total_sales DESC) AS rnk
334     FROM product_sales
335 )
336 SELECT * FROM ranked WHERE rnk = 1;
337
338 /* -----
339
340
341
342 /*=====
343 END OF ASSIGNMENT
344 Intern : Arati Thorat
345 =====*/
346
```